

Executive Summary

The Nepalese Student Association (NSA) Website is a web-based management platform developed to improve communication, transparency, and engagement among students and administrators of the Nepalese Student Association. Traditional methods of sharing information through social media platforms and messaging applications often resulted in fragmented communication, missed announcements, and inefficient management of association activities. To address these challenges, the NSA Website was developed as a centralized platform that integrates event management, notice dissemination, survey collection, job opportunity sharing, and user management functionalities.

The system was designed and implemented using modern web technologies including React.js for the frontend, Node.js and Express.js for the backend, and MySQL as the database management system. The platform was deployed using Railway cloud services to provide accessibility and reliability. Security features such as JSON Web Token (JWT) authentication, password hashing using bcrypt, and email-based OTP verification were implemented to ensure secure user access and account management.

The development process followed Software Engineering principles including requirements analysis, architectural design, detailed design, implementation, testing, and deployment. Various UML artifacts including use case diagrams, class diagrams, sequence diagrams, and an entity relationship diagram were created to support system analysis and design. Comprehensive testing was conducted to validate functional requirements and system reliability.

The completed system provides students with convenient access to events, notices, surveys, and job opportunities while enabling administrators to efficiently manage association operations. The project demonstrates the practical application of software engineering methodologies and modern web development technologies in solving real-world organizational challenges.

Overall, the NSA Website successfully fulfills its primary objectives and establishes a scalable foundation for future enhancements and continued digital transformation within the Nepalese Student Association.

Title Page

Executive Summary

Table of Contents

List of Figures

List of Tables

1. Introduction

1.1 Purpose and Scope

1.2 Product Overview

1.3 Structure of the Document

1.4 Terms, Acronyms and Abbreviations

2. Project Management Plan

2.1 Project Organization

2.2 Lifecycle Model Used

2.3 Risk Analysis

2.4 Hardware and Software Resource Requirements

2.5 Deliverables and Schedule

2.6 Monitoring, Reporting and Controlling Mechanisms

2.7 Professional Standards

2.8 Configuration Management

2.9 Impact on Individuals and Organizations

3. Requirement Specifications

3.1 Stakeholders for the System

3.2 Use Case Model

3.2.1 Graphic Use Case Model

3.2.2 Textual Description for Each Use Case

3.3 Rationale for Use Case Model

3.4 Non-Functional Requirements

4. Architecture

4.1 Architectural Style Used

4.2 Architectural Model

4.3 Technology, Software and Hardware Used

4.4 Rationale for Architectural Style and Model

5. Design

5.1 GUI Design

5.2 Static Model – Class Diagram

5.3 Dynamic Model – Sequence Diagrams

5.4 Rationale for Detailed Design Model

5.5 Traceability from Requirements to Design

6. Test Plan

6.1 Requirements-Based System Level Test Cases

6.2 Traceability of Test Cases to Use Cases

6.3 Techniques Used for Test Generation

6.4 Assessment of the Goodness of the Test Suite

Acknowledgment

References

Figure 3.1 Context Diagram

Figure 3.2 Administrator Use Case Diagram

Figure 3.3 Student Use Case Diagram

Figure 4.1 Three-Tier Architecture Diagram

Figure 5.1 Login Page Interface

Figure 5.2 Registration Page Interface

Figure 5.3 Student Dashboard Interface

Figure 5.4 Administrator Dashboard Interface

Figure 5.5 Event Management Interface

Figure 5.6 Notice Management Interface

Figure 5.7 Job Portal Interface

Figure 5.8 Survey Management Interface

Figure 5.9 Entity Relationship Diagram

Figure 5.10 Class Diagram

Figure 5.11 Sequence Diagram – User Login

Figure 5.12 Sequence Diagram – Admin Creates Event

Figure 5.13 Sequence Diagram – Student Submits Survey

Figure 5.14 Sequence Diagram – Admin Posts Job

Table 1.1 Terms, Acronyms and Abbreviations

Table 2.1 Team Roles and Responsibilities

Table 2.2 Risk Analysis Matrix

Table 2.3 Hardware Requirements

Table 2.4 Software Requirements

Table 2.5 Project Deliverables

Table 3.1 Stakeholders of the System

Table 3.2 Functional Requirements

Table 3.3 Non-Functional Requirements

Table 5.1 Requirements to Design Traceability Matrix

Table 6.1 System Test Cases

Table 6.2 Use Case to Test Case Traceability Matrix

Table 6.3 Test Coverage Summary

Chapter 1

Introduction

1.1 Purpose and Scope

The Nepalese Student Association (NSA) Website is a web-based platform developed to improve communication, information sharing, event management, and student engagement among Nepalese students at Dongshin University. Traditionally, important information regarding events, notices, surveys, job opportunities, and financial activities was shared through social media platforms and messaging applications, which often resulted in scattered information, missed announcements, and inefficient communication.

The purpose of this project is to provide a centralized digital platform where students and administrators can efficiently manage and access association-related information. The system enables administrators to create and manage events, notices, job opportunities, surveys, budgets, and user accounts, while students can access these services through a secure and user-friendly interface.

The scope of the project includes:

- User registration and authentication
- Role-based access control (Admin and Student)
- Event management system
- Notice board management
- Job opportunity portal
- Student survey system
- Budget transparency module
- User management system
- Secure database storage and retrieval
- Responsive web interface accessible from multiple devices

The system does not include payment processing, direct messaging, or mobile application support in the current version.

1.2 Product Overview

The NSA Website is a full-stack web application designed to serve as the official digital platform for the Nepalese Student Association. The system follows a three-tier client-server architecture consisting of a React frontend, Node.js/Express backend, and MySQL database.

Key Capabilities

i. User Management

Students can register and create accounts using their academic information. Administrators can manage user accounts and assign permissions.

ii. Authentication and Security

The platform provides secure login functionality using encrypted passwords and JSON Web Tokens (JWT). Email-based OTP verification is also integrated for account validation and password recovery.

iii. Event Management

Administrators can create, update, and delete events. Students can view upcoming events through a centralized event dashboard.

iv. Notice Management

Administrators can publish official announcements through a digital notice board. Students can access notices in real time.

v. Job Portal

Administrators can post employment opportunities, while students can browse and apply for available positions.

vi. Survey System

Administrators can create surveys to collect student feedback, and students can participate in surveys directly through the platform.

vii. Budget Transparency

Administrators can maintain budget records, and students can access financial summaries to promote transparency within the association.

Typical Usage Scenario

A student registers for the NSA Website and verifies their email address through an OTP.

After logging in, the student can:

- View upcoming events
- Read official notices
- Browse available jobs
- Participate in surveys
- Review budget summaries

An administrator logs into the system and can:

- Manage users
- Create and update events
- Publish notices
- Post job opportunities
- Create surveys
- Manage budget information

The platform serves as a centralized hub for all association activities and communications.

1.3 Structure of the Document

This report is organized into six major chapters:

Chapter 1 – Introduction

Provides the project purpose, scope, product overview, and document organization.

Chapter 2 – Project Management Plan

Describes project organization, development methodology, resource allocation, risks, schedules, and project management activities.

Chapter 3 – Requirement Specifications

Presents stakeholders, use case models, functional requirements, and non-functional requirements.

Chapter 4 – Architecture

Explains the system architecture, technology stack, architectural design decisions, and implementation environment.

Chapter 5 – Design

Describes the graphical user interface, class diagrams, sequence diagrams, database design, and traceability from requirements to design.

Chapter 6 – Test Plan

Provides system-level test cases, testing techniques, test coverage analysis, and traceability between requirements and testing activities.

The report concludes with acknowledgments and references used throughout the project.

1.4 Terms, Acronyms, and Abbreviations

Term	Description
NSA	Nepalese Student Association
GUI	Graphical User Interface
JWT	JSON Web Token
RBAC	Role-Based Access Control
API	Application Programming Interface
DBMS	Database Management System
MySQL	Relational Database Management System used in the project
OTP	One-Time Password
SMTP	Simple Mail Transfer Protocol
CRUD	Create, Read, Update, Delete
FE	Frontend
BE	Backend
UML	Unified Modeling Language

ER Diagram	Entity Relationship Diagram
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
React	Frontend JavaScript Library
Node.js	Server-side JavaScript Runtime Environment
Express.js	Web Framework for Node.js
Vite	Frontend Build Tool
Git	Version Control System
GitHub	Source Code Repository Platform

Chapter 2

Project Management Plan

2.1 Project Organization

The NSA Website project was developed as part of the Software Engineering course (ITS10-02) at Dongshin University. The project team was responsible for gathering requirements, designing the system architecture, implementing the software solution, testing the system, and preparing project documentation.

The project was organized using a collaborative team structure where responsibilities were distributed among team members according to system modules and expertise.

Team Roles and Responsibilities

Role	Responsibility
Project Manager	Planning, scheduling, coordination, progress monitoring, and report preparation
Backend Developer	Development of REST APIs, database integration, authentication, and business logic
Frontend Developer	User interface development, page navigation, and user experience design
Database Designer	Database design, schema creation, and data management
Tester	Functional testing, bug identification, and validation
Documentation Manager	Preparation of SRS, design documents, RTM, and final report

Although responsibilities were distributed, all members participated in discussions, testing, and design reviews throughout the development process.

Organizational Structure

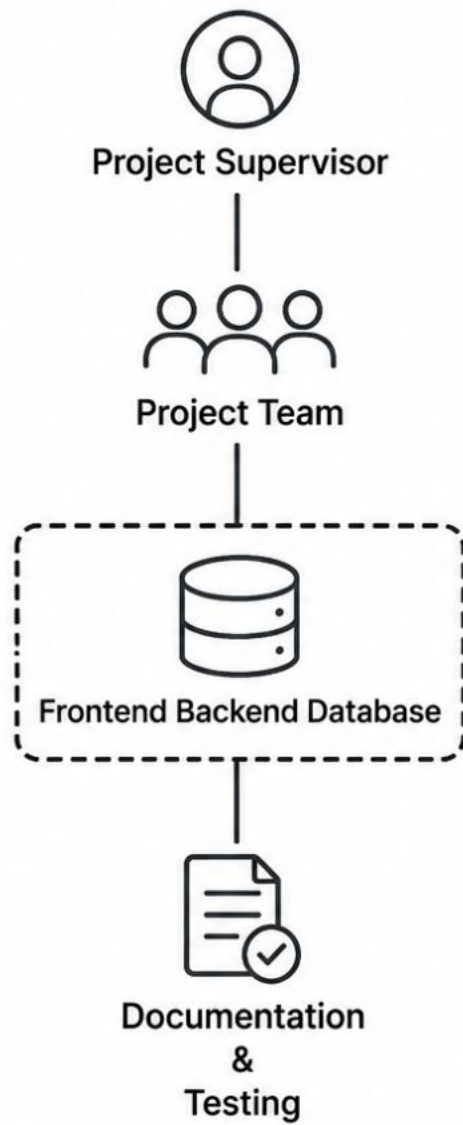


Figure 1: Organizational Structure

2.2 Lifecycle Model Used

The project adopted the Agile Software Development Model.

Agile was selected because it allows continuous development, frequent testing, and iterative improvement of software features. Since requirements evolved during the development process, Agile provided flexibility for incorporating feedback and modifications.

Agile Development Process

The development process was divided into several iterations:

Iteration 1 – Requirements Analysis

- Stakeholder identification
- Requirement gathering
- SRS preparation
- Use case development

Iteration 2 – System Design

- Architecture design
- Database design
- Class diagrams
- Sequence diagrams

Iteration 3 – Backend Development

- Authentication system
- User management APIs
- Event APIs
- Notice APIs
- Job APIs
- Survey APIs

Iteration 4 – Frontend Development

- Registration page
- Login page
- Dashboard
- Event management interface
- Job portal
- Survey system

Iteration 5 – Testing and Integration

- Functional testing
- API testing
- Database testing
- Bug fixing

Iteration 6 – Deployment and Documentation

- Railway deployment
- Database migration
- Final report preparation

Advantages of Agile for this Project

- Supports changing requirements.
- Enables early testing.
- Improves collaboration.
- Allows continuous improvement.
- Facilitates incremental delivery.

2.3 Risk Analysis

Risk management was performed throughout the project lifecycle to identify and mitigate potential issues.

Risk Assessment Table

Risk	Probability	Impact	Mitigation Strategy
Requirement changes	Medium	High	Regular stakeholder discussions
Database connection failures	High	High	Backup database configuration and testing
Deployment issues	High	Medium	Continuous deployment testing on Railway
Team communication problems	Medium	Medium	Weekly progress meetings
Security vulnerabilities	Medium	High	JWT authentication and password hashing
Data loss	Low	High	Regular database backups
Schedule delays	Medium	Medium	Milestone tracking and task prioritization
Third-party email service failure	Medium	Medium	Alternative email service providers

Risk Mitigation Examples

During deployment, the team experienced package dependency conflicts and SMTP email delivery issues. These risks were mitigated by updating package-lock files, using cloud-hosted databases, and integrating Brevo email services as an alternative communication solution.

2.4 Hardware and Software Resource Requirements

Hardware Requirements

Development Environment

Hardware	Specification
Processor	Intel Core i5 or higher
RAM	8 GB minimum
Storage	256 GB SSD minimum
Internet	Broadband connection
Device	Laptop/Desktop

Deployment Environment

Component	Specification
Railway Cloud Platform	Application Hosting
Railway MySQL Service	Cloud Database
Cloud Storage	Application Data

Software Requirements

Software	Purpose
Visual Studio Code	Source code development
Node.js	Backend runtime
Express.js	Web framework
React.js	Frontend development
Vite	Frontend build tool

MySQL	Database management
Railway	Cloud deployment
Git	Version control
GitHub	Source code repository
Postman	API testing
Figma	UI design
MySQL Workbench	Database administration

New Technologies Learned During the Project

Technology	Learning Outcome
React.js	Component-based frontend development
Express.js	RESTful API development
JWT Authentication	Secure user authentication
Railway Deployment	Cloud application hosting
MySQL Integration	Database connectivity and management
Git & GitHub	Collaborative version control
Figma	Interface prototyping
SMTP/Brevo API	Email and OTP services

The project provided valuable practical experience in full-stack web application development and cloud deployment technologies.

2.5 Deliverables and Schedule

Project Deliverables

Deliverable	Status
Software Requirements Specification (SRS)	Completed
Context Diagram	Completed
Use Case Diagram	Completed
ER Diagram	Completed
Class Diagram	Completed
Sequence Diagrams	Completed
Frontend Application	Completed
Backend APIs	Completed
Database Schema	Completed
Testing Report	Completed
Final Report	Completed
Deployment on Railway	Completed

Project Schedule

Phase	Duration
Requirement Analysis	Week 1-2
System Design	Week 3-4
Database Design	Week 5
Backend Development	Week 6-8
Frontend Development	Week 9-11
Integration Testing	Week 12
Deployment	Week 13
Documentation	Week 14

2.6 Monitoring, Reporting, and Controlling Mechanisms

Project progress was monitored through regular team meetings and milestone reviews.

Monitoring Techniques

- Weekly progress discussions
- Task completion tracking
- GitHub commit monitoring
- Feature completion reviews
- API testing reports

Reporting Methods

- Weekly status reports
- GitHub commit history
- Project demonstrations
- Design review sessions

Control Mechanisms

- Version control using Git
- Code review before merging
- Requirement Traceability Matrix (RTM)
- Testing and bug tracking

These mechanisms ensured project quality and schedule adherence.

2.7 Professional Standards

The project followed recognized software engineering standards and best practices.

Standards Applied

Coding Standards

- Consistent naming conventions
- Modular programming
- Code documentation

Security Standards

- Password hashing using bcrypt
- JWT authentication
- Input validation
- Protected API routes

Documentation Standards

- IEEE Software Engineering Report Format
- UML Modeling Standards
- Requirement Traceability Matrix (RTM)

Database Standards

- Relational database design
- Foreign key constraints
- Data integrity enforcement

2.8 Evidence That All Artifacts Were Placed Under Configuration Management

Configuration management was implemented using Git and GitHub.

Repository Management

The NSA Website source code, documentation, diagrams, and reports were stored in a centralized GitHub repository.

Artifacts under version control included:

- Source code
- SRS document
- Design diagrams
- Testing artifacts
- Final report
- Database scripts

Configuration Management Activities

- Version tracking
- Branch management
- Commit history maintenance
- Code merging
- Backup storage

GitHub provided traceability and ensured all project artifacts remained synchronized throughout development.

2.9 Impact of the Project on Individuals and Organizations

Impact on Students

The NSA Website improves communication and information accessibility for students by providing a centralized platform for events, notices, jobs, and surveys.

Benefits include:

- Faster access to information
- Increased student engagement
- Better employment opportunities
- Improved communication
- Increased transparency

Impact on the Nepalese Student Association

The platform modernizes association operations by replacing manual communication methods with a centralized digital system.

Benefits include:

- Efficient event management
- Simplified notice distribution
- Better survey collection
- Improved budget transparency
- Reduced administrative workload

Impact on Society

The project demonstrates how digital technologies can enhance community organizations and student associations. It promotes transparency, accountability, and accessibility while encouraging digital transformation in educational communities.

Educational Impact

The project enabled team members to gain practical experience in:

- Software Engineering
- Web Development

- Database Systems
- Cloud Deployment
- Team Collaboration
- Project Management

The experience significantly improved both technical and professional skills, preparing team members for future software development careers.

Chapter 3

Requirement Specifications

3.1 Stakeholders for the System

Stakeholders are individuals or organizations that interact with or are affected by the NSA Website system.

Primary Stakeholders

i. Nepalese Student Association (NSA) Administrators

Administrators are responsible for managing the entire system. They create events, publish notices, manage surveys, oversee job postings, and maintain user accounts.

Responsibilities:

- Manage events
- Post notices
- Create surveys
- Approve content
- Manage users
- Monitor system activities

ii. Students

Students are the primary users of the system. They use the platform to access announcements, register for events, participate in surveys, and browse job opportunities.

Responsibilities:

- Register accounts
- Login to the system
- View notices
- Participate in surveys
- Apply for jobs
- Update profile information

Secondary Stakeholders

i. Employers

Employers provide job opportunities for students through the Job Portal module.

Responsibilities:

- Submit job postings
- Recruit students
- Communicate opportunities

ii. University Community

The broader university community benefits from improved communication and event management.

Benefits:

- Easier information dissemination
- Increased student participation
- Better organization of activities

iii. Development Team

The development team designs, implements, tests, and maintains the system.

Responsibilities:

- Software development
- Bug fixing
- System deployment
- Maintenance

Stakeholder Relationship Diagram

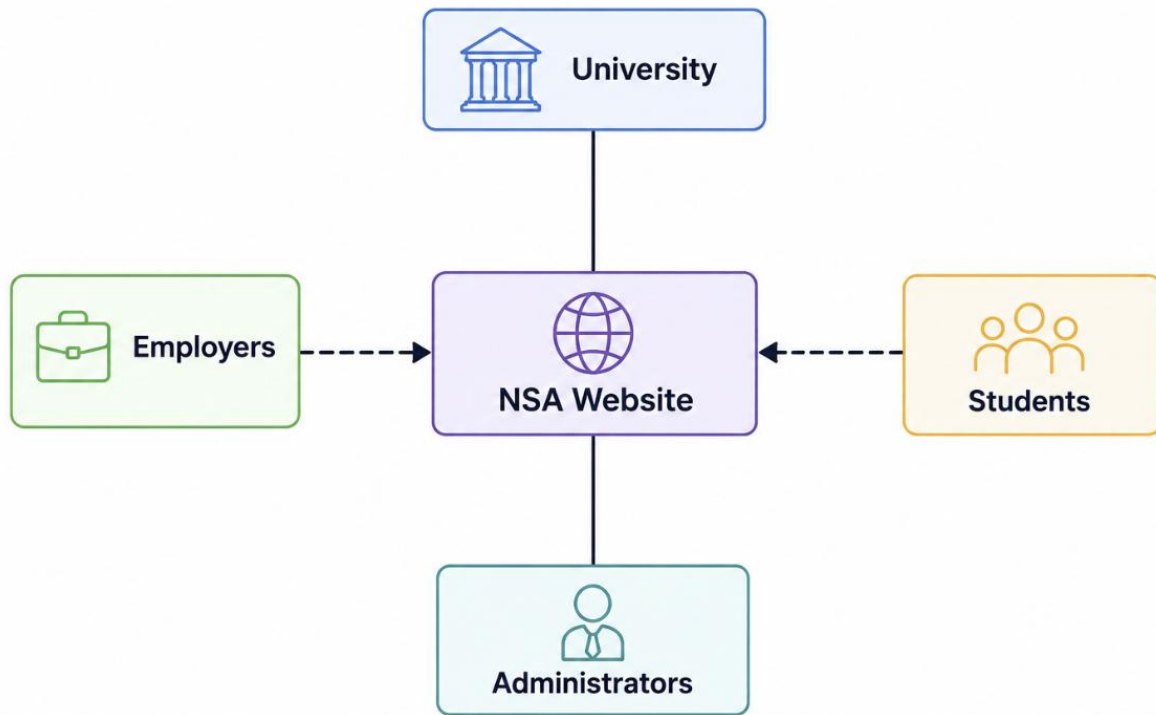


Figure 2: Stakeholder Relationship Diagram

3.2 Use Case Model

The use case model describes the interactions between users and the NSA Website system.

Figure 3.2.1: NSA Website Use Case Diagram

The use case model contains two actors:

i. Student

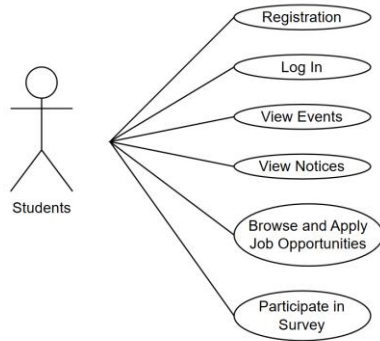


Figure 3: Use case diagram-Student

Student can:

- Register Account
- Verify OTP
- Login
- Manage Profile
- View Events
- View Notices
- Browse Jobs
- Submit Surveys
- Logout

ii. Administrator

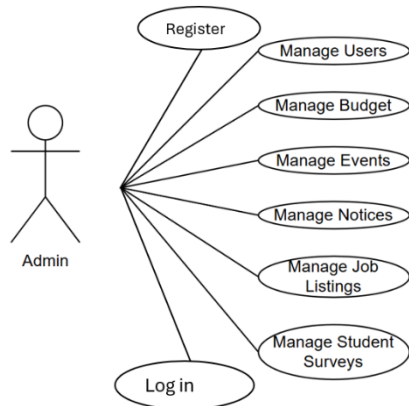


Figure 4: Use case diagram-Admin

Administrator can:

- Login
- Manage Events
- Manage Notices
- Manage Surveys
- Manage Jobs
- Manage Users
- Manage Budget Records
- Logout

3.2.2 Textual Description for Each Use Case

UC-01 User Registration

Item	Description
Use Case ID	UC-01
Use Case Name	User Registration
Primary Actor	Student
Goal	Create a new user account
Preconditions	User is not registered
Postconditions	Account successfully created

Main Flow

1. User opens registration page.
2. User enters personal information.
3. User requests OTP verification.
4. System sends OTP to email.
5. User enters OTP.
6. System validates OTP.
7. Account is created successfully.

Alternative Flow

- Invalid OTP entered.
- System displays error message.

UC-02 User Login

Item	Description
Use Case ID	UC-02
Use Case Name	User Login
Primary Actor	Student/Admin
Goal	Access the system
Preconditions	User account exists
Postconditions	User is authenticated

Main Flow

1. User enters email and password.
2. System validates credentials.
3. JWT token is generated.
4. User accesses dashboard.

Alternative Flow

- Invalid credentials.
- System displays login error.

UC-03 Manage Events

Item	Description
Use Case ID	UC-03
Use Case Name	Manage Events
Primary Actor	Administrator
Goal	Create, update, and delete events

Main Flow

1. Admin opens Event Management.
2. Admin creates or edits event details.

3. System saves event information.
4. Students can view events.

UC-04 View Events

Item	Description
Use Case ID	UC-04
Use Case Name	View Events
Primary Actor	Student
Goal	Access event information

Main Flow

1. Student opens Events page.
2. System displays event list.
3. Student views event details.

UC-05 Manage Notices

Item	Description
Use Case ID	UC-05
Use Case Name	Manage Notices
Primary Actor	Administrator
Goal	Publish announcements

Main Flow

1. Admin creates notice.
2. Notice is saved.
3. Notice becomes visible to users.

UC-06 View Notices

Item	Description
Use Case ID	UC-06
Use Case Name	View Notices
Primary Actor	Student
Goal	Read announcements

Main Flow

1. Student opens Notice Board.
2. System displays notices.
3. Student reads details.

UC-07 Manage Surveys

Item	Description
Use Case ID	UC-07
Use Case Name	Manage Surveys
Primary Actor	Administrator
Goal	Create surveys

Main Flow

1. Admin creates survey questions.
2. Survey is published.
3. Students can participate.

UC-08 Submit Survey

Item	Description
Use Case ID	UC-08
Use Case Name	Submit Survey
Primary Actor	Student
Goal	Participate in surveys

Main Flow

1. Student opens survey.
2. Student answers questions.
3. System stores responses.

UC-09 Manage Job Listings

Item	Description
Use Case ID	UC-09
Use Case Name	Manage Job Listings
Primary Actor	Administrator
Goal	Publish employment opportunities

Main Flow

1. Admin creates job listing.
2. Job is stored.
3. Students can browse opportunities.

UC-10 Browse Jobs

Item	Description
Use Case ID	UC-10
Use Case Name	Browse Jobs
Primary Actor	Student
Goal	Search available jobs

Main Flow

1. Student opens Job Portal.
2. System displays available jobs.
3. Student reviews job details.

UC-11 Manage Budget Records

Item	Description
Use Case ID	UC-11
Use Case Name	Manage Budget Records
Primary Actor	Administrator
Goal	Maintain financial records

Main Flow

1. Admin enters budget information.
2. System stores budget records.
3. Reports become available.

UC-12 Manage User Accounts

Item	Description
Use Case ID	UC-12
Use Case Name	Manage Users
Primary Actor	Administrator
Goal	Manage system users

Main Flow

1. Admin accesses user management.
2. Admin updates user information.
3. System saves changes.

3.3 Rationale for the Use Case Model

The use case model was developed to clearly identify the functional requirements and interactions between system users and the NSA Website.

The model separates responsibilities between students and administrators, ensuring proper access control and security. Students focus on consuming information and participating in activities, while administrators manage content and system operations.

The selected use cases directly support the primary objectives of the project:

- Improve communication within the Nepalese Student Association.
- Centralize event management.
- Provide employment opportunities.
- Enable survey collection.
- Improve transparency and information accessibility.

The use case model also served as the foundation for system design, class diagrams, sequence diagrams, database design, and testing activities.

3.4 Non-Functional Requirements

The NSA Website must satisfy several quality requirements in addition to functional requirements.

Performance Requirements

Requirement ID	Description
NFR-01	System shall respond to user requests within 3 seconds under normal conditions.
NFR-02	System shall support at least 100 concurrent users.
NFR-03	Database queries shall complete within 2 seconds.

Security Requirements

Requirement ID	Description
NFR-04	User passwords shall be encrypted using bcrypt hashing.
NFR-05	Authentication shall be implemented using JWT tokens.
NFR-06	Unauthorized users shall not access protected resources.

Reliability Requirements

Requirement ID	Description
NFR-07	System availability shall be at least 99%.
NFR-08	Database backups shall be maintained regularly.
NFR-09	System shall recover from failures without data loss.

Usability Requirements

Requirement ID	Description
NFR-10	User interface shall be intuitive and easy to navigate.
NFR-11	New users shall learn basic functions within 15 minutes.
NFR-12	System shall provide meaningful error messages.

Maintainability Requirements

Requirement ID	Description
NFR-13	System architecture shall support future enhancements.
NFR-14	Source code shall follow modular programming practices.
NFR-15	Documentation shall be maintained throughout development.

Portability Requirements

Requirement ID	Description
NFR-16	System shall operate on modern web browsers.
NFR-17	System shall support Windows, Linux, and macOS platforms.
NFR-18	Cloud deployment shall be supported through Railway.

Chapter 4

Architecture

4.1 Architectural Style(s) Used

The NSA Website follows a **Three-Tier Client-Server Architecture** combined with a **RESTful API Architecture**. This architectural style separates the presentation layer, business logic layer, and data layer, making the system scalable, maintainable, and easier to develop.

Three-Tier Architecture

The architecture consists of three layers:

1. Presentation Layer (Frontend)

The presentation layer provides the graphical user interface (GUI) through which users interact with the system.

Responsibilities:

- Display web pages
- Collect user inputs
- Send requests to the backend
- Display server responses

Technology Used:

- React.js
- Bootstrap 5
- Axios
- React Router DOM

2. Business Logic Layer (Backend)

The business logic layer processes user requests and implements system functionality.

Responsibilities:

- User authentication
- OTP verification
- Event management
- Notice management
- Job portal operations
- Survey processing
- Budget management

Technology Used:

- Node.js
- Express.js
- JWT Authentication
- Nodemailer/Brevo Email Service

3. Data Layer (Database)

The data layer stores and retrieves system data.

Responsibilities:

- Store user accounts
- Store events
- Store notices
- Store surveys
- Store job listings
- Store budget records

Technology Used:

- MySQL
- Railway Cloud Database

4.2 Architectural Model

System Architecture Diagram

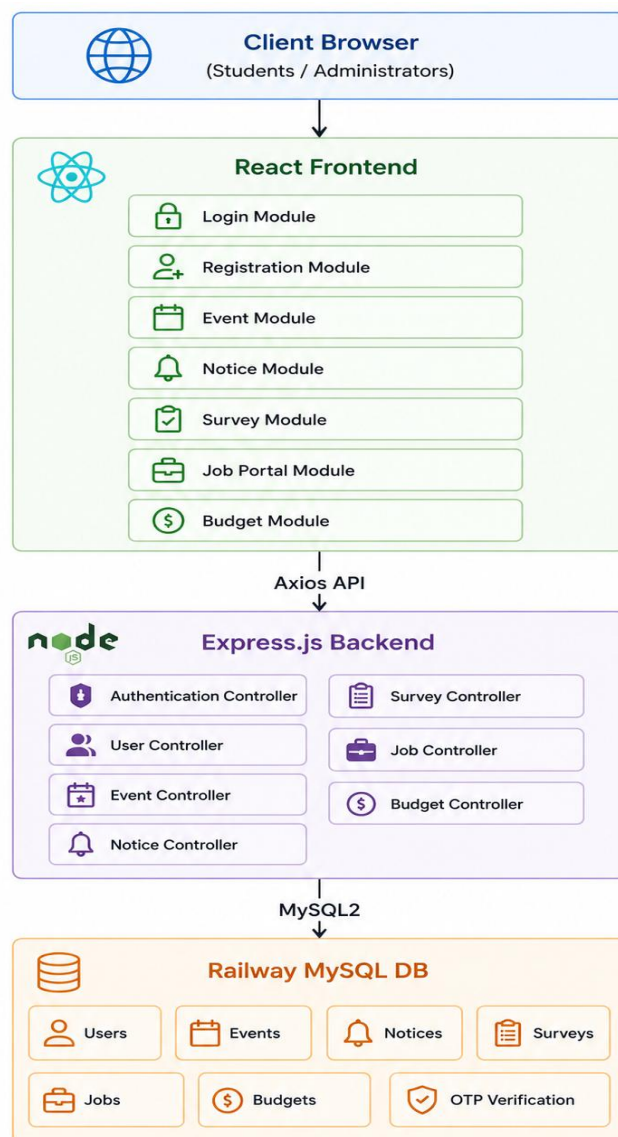


Figure 5: Overall System Architecture

Request Processing Flow

1. User accesses NSA Website through browser.
2. React frontend sends API request using Axios.
3. Express backend receives request.
4. Controller processes business logic.
5. MySQL database executes query.
6. Database returns result.
7. Backend sends JSON response.
8. Frontend updates interface.

Authentication Architecture

The authentication mechanism uses JSON Web Tokens (JWT).

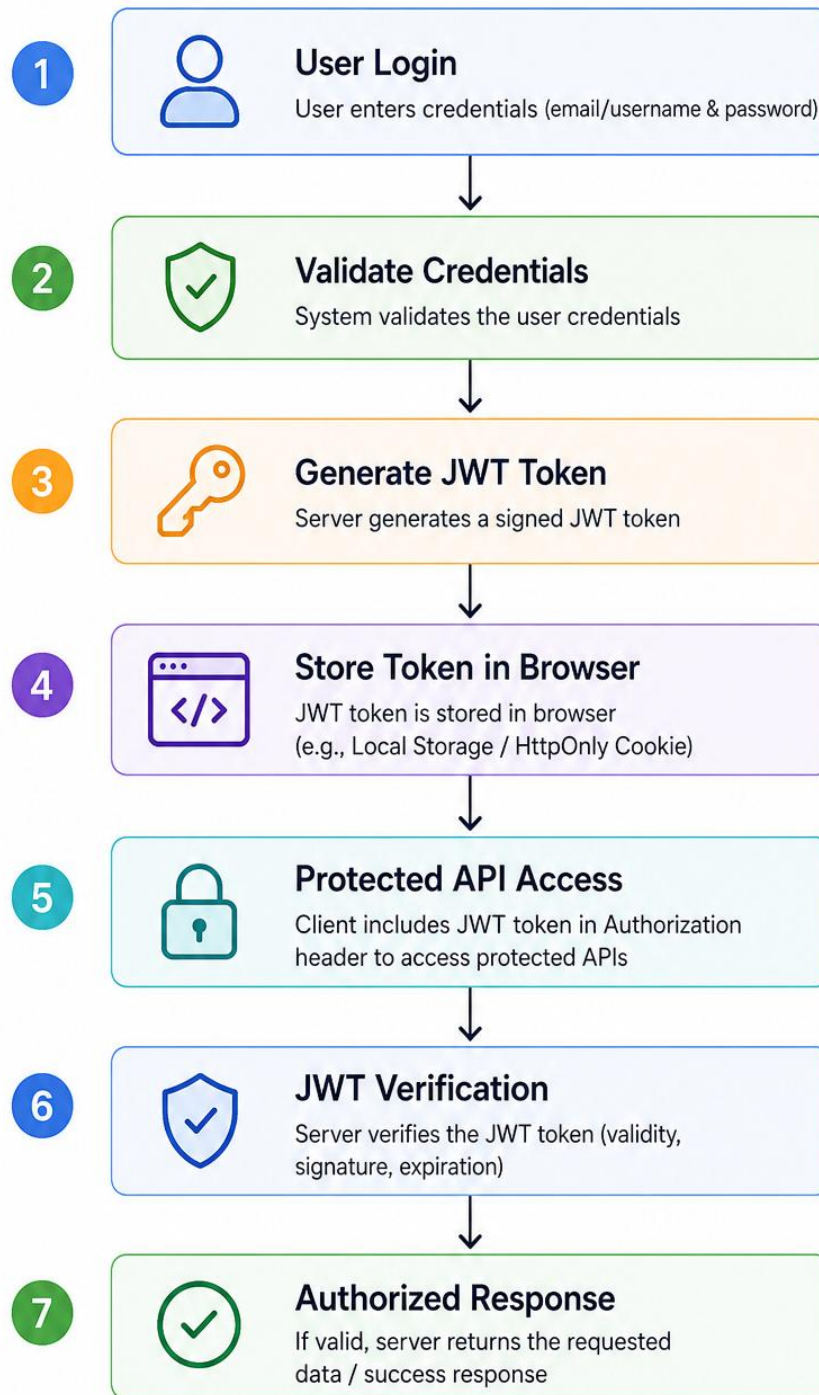


Figure 6: Authentication Architecture

Email Verification Architecture

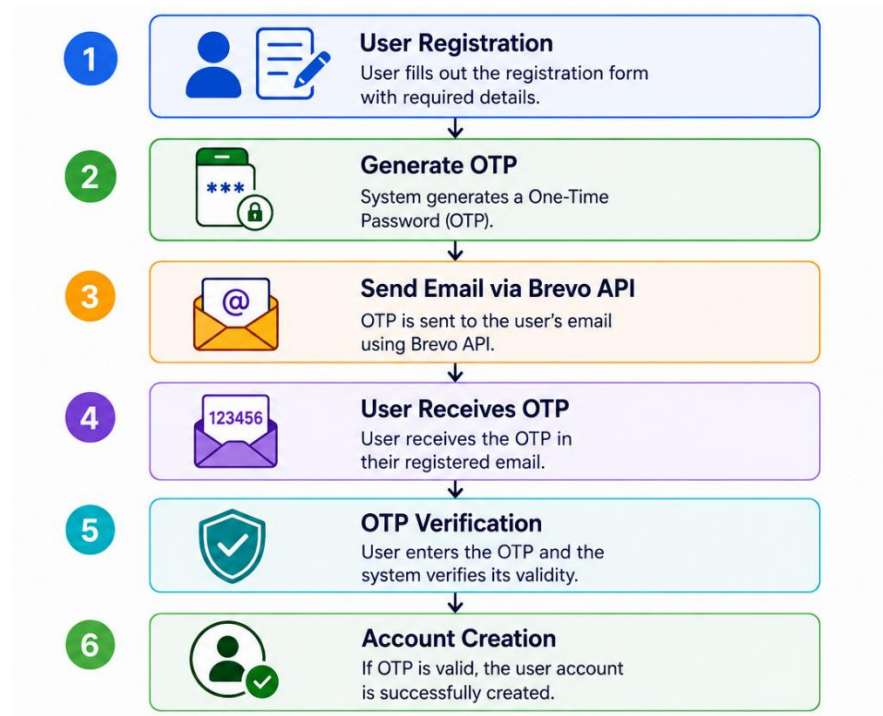


Figure 7: OTP Verification Process

4.3 Technology, Software, and Hardware Used

Software Technologies

Category	Technology
Frontend Framework	React.js
Styling Framework	Bootstrap 5
Backend Runtime	Node.js
Backend Framework	Express.js
Database	MySQL
Authentication	JWT
Password Encryption	bcrypt
API Communication	Axios

Email Service	Brevo API
Version Control	Git
Repository Hosting	GitHub
Cloud Hosting	Railway
IDE	Visual Studio Code
Database Management	MySQL Workbench
Testing Tools	Postman
Documentation	Microsoft Word

Hardware Requirements

Development Environment

Component	Specification
Processor	Intel Core i5 or higher
RAM	Minimum 8 GB
Storage	256 GB SSD
Internet	Broadband Connection
Operating System	Windows 11

Hardware Used by Team Members

Member	Hardware Used
Sabin	Windows Laptop, Intel Processor, VS Code
Milan	Windows Laptop
Ramuna	Windows Laptop
Anish	Windows Laptop
Sumit	Windows Laptop

New Technologies Learned During the Project

Sabin

- Railway Cloud Deployment
- JWT Authentication
- MySQL Cloud Integration
- Brevo Email API
- React Router
- Git Branch Management

Milan

- React Components
- Frontend State Management
- Bootstrap Integration

Ramuna

- MySQL Database Design
- REST API Development

Anish

- Software Testing
- Documentation Preparation

Sumit

- React Components
- Frontend State Management
- Bootstrap Integration

4.4 Rationale for Architectural Style and Model

The Three-Tier Architecture was selected because it provides clear separation of concerns between the user interface, business logic, and database layers.

Advantages

i.Maintainability

Changes in one layer do not significantly affect other layers. For example, frontend modifications can be implemented without changing the database structure.

ii.Scalability

Additional modules such as attendance tracking, membership management, or mobile applications can be integrated easily.

iii.Security

Sensitive information is processed on the server side. JWT authentication and password encryption enhance security.

iv.Reusability

Backend APIs can be reused by future mobile applications or external systems.

v.Cloud Compatibility

The architecture supports deployment on cloud platforms such as Railway, making the system accessible from anywhere.

Why React + Express + MySQL?

React

- Fast user interface rendering
- Component-based architecture
- Reusable code structure

Express

- Lightweight backend framework
- Easy API development

- High compatibility with React

MySQL

- Reliable relational database
- Easy integration with Node.js
- Strong support for structured data

Railway

- Simplified deployment process
- Cloud database hosting
- Continuous deployment through GitHub

Chapter 5

Design

5.1 GUI (Graphical User Interface) Design

The Graphical User Interface (GUI) of the NSA Website was designed to provide a modern, responsive, and user-friendly experience for both students and administrators. The interface was initially prototyped using Figma and later implemented using React.js and Bootstrap.

The design emphasizes simplicity, accessibility, and ease of navigation while maintaining a professional appearance suitable for an academic organization.

Design Principles

The GUI was developed according to the following principles:

- Simplicity and ease of use
- Consistent navigation structure
- Responsive design for multiple devices
- Clear visual hierarchy
- Minimal learning curve for users

Main Interfaces

i. Login Interface

The Login Page provides secure access to the platform for students and administrators. Users enter their email address and password, which are validated by the backend authentication service.

Features:

- Email authentication
- Password authentication
- Forgot Password functionality
- Secure JWT-based login

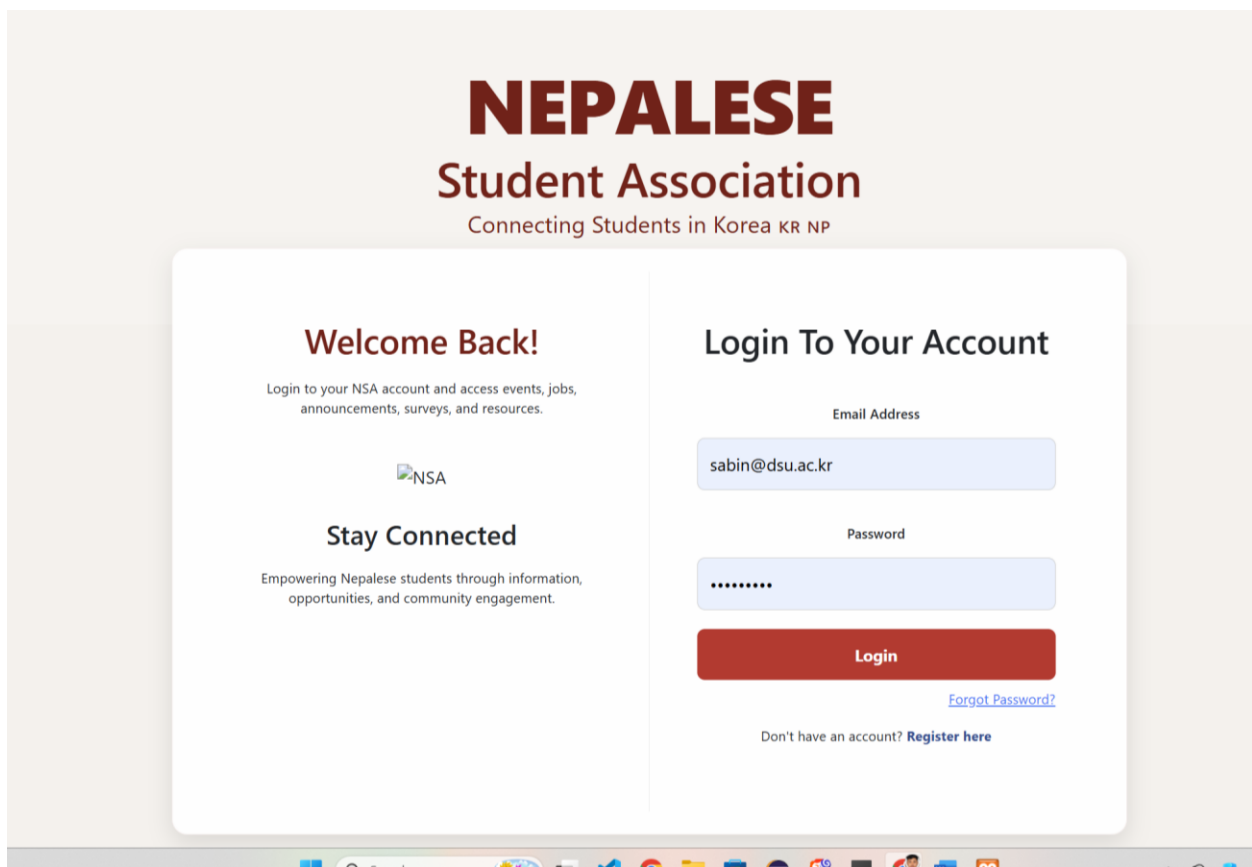


Figure 8: Login Page Screenshot

ii. Registration Interface

The Registration Page allows students to create accounts by entering personal and academic information. Email verification using OTP ensures account authenticity.

Features:

- Student information entry
- OTP verification
- Password confirmation
- Input validation

Create Your Account
Join the Nepalese Student Association and connect with students across Korea.

Be a Part of NSA
Create an account to access resources, apply for jobs, join events, and stay updated with announcements.

- Access student resources
- Join events and programs
- Apply for jobs and opportunities
- Get important announcements

Registration Form

Full Name * sabin@dsu.ac.kr

Student ID *

Phone Number Nationality

Department Course / Major

Enrollment Year

Send OTP Enter OTP **Verify OTP**

☐ I agree to the Terms & Conditions and Privacy Policy

Create Account

Already have an account? [Login](#)

Figure 9: Registration Page Screenshot

iii. Student Dashboard

The Student Dashboard serves as the central hub for students after successful login.

Features:

- Event announcements
- Notice board access
- Job opportunities
- Survey participation
- User profile management

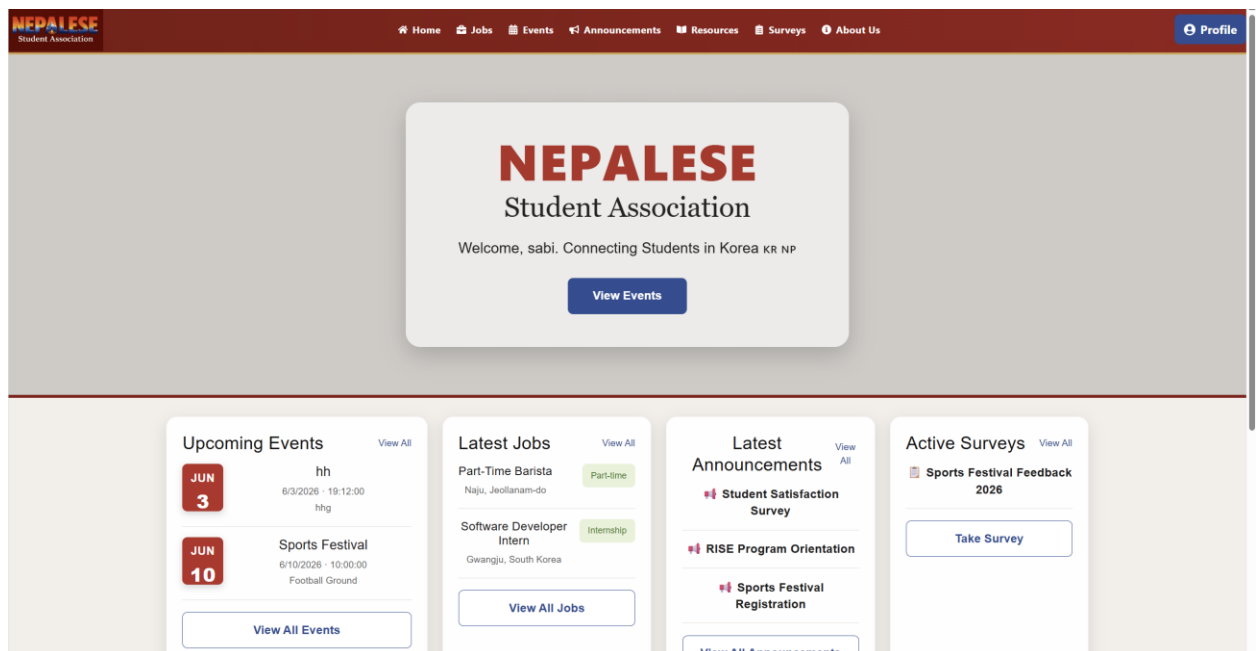


Figure 10: Student Dashboard Screenshot

iv. Administrator Dashboard

The Administrator Dashboard provides management tools for association executives.

Features:

- Event Management
- Notice Management
- Survey Management
- Job Management
- User Management
- Budget Management

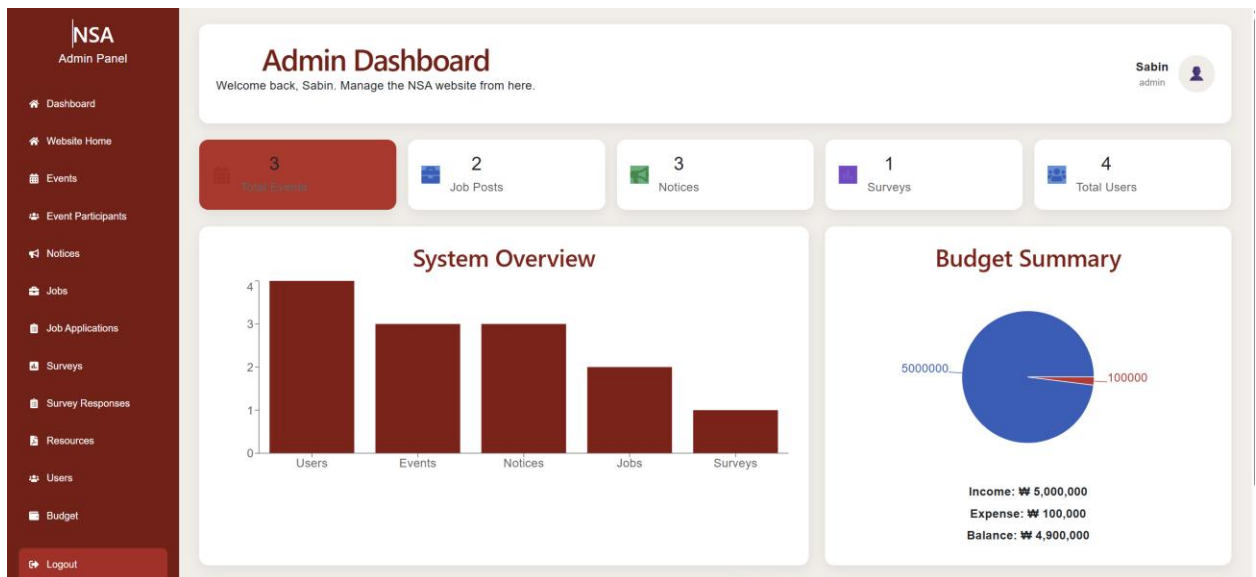


Figure 11: Administrator Dashboard Screenshot

v. Event Management Interface

Administrators can create, edit, and delete events through an intuitive management panel.

Features:

- Event title
- Event description
- Date and time
- Location details
- Event status

Event Management

Create, view, update, and delete NSA events.

Create Event

Event Title

Enter event title

Description

Enter event description

Event Date

mm/dd/yyyy

Event Time

HH:MM AM

Location

Enter location

Create Event

All Events

ID	Title	Date	Time	Location	Actions
8	hh	2026-06-03	19:12:00	hhg	EditDelete
1	Sports Festival	2026-06-10	10:00:00	Football Ground	EditDelete
4	Dashain Festival	2026-10-20	07:00:00	Together Hall	EditDelete

Figure 12: Event Management Interface

vi. Notice Board Interface

The Notice Board allows administrators to publish announcements and students to access information.

Features:

- Notice title
- Notice content
- Publication date
- Search functionality

The screenshot displays the 'Notice Management' interface. At the top, a dark red header bar contains the title 'Notice Management' and a sub-header 'Create, update, and manage NSA announcements.' Below the header, the interface is divided into two main sections. On the left is a 'Create Notice' form with a red '+' icon. It includes a 'Notice Title' field with the placeholder 'Enter notice title' and a 'Notice Content' text area with the placeholder 'Write notice content here...'. A red 'Create Notice' button is at the bottom of the form. On the right is the 'All Notices' section, which shows 'Showing 3 of 3 notices'. It features a search bar with the placeholder 'Search notices...'. Below the search bar, three notice cards are displayed. Each card has a red 'Notice' icon, a title, a date, and 'Edit' and 'Delete' buttons. The first card is titled 'Student Satisfaction Survey' with the date '6/6/2026' and the content 'Evaluate academic services, facilities, and student support provided by the university.' The second card is titled 'RISE Program Orientation' with the date '6/6/2026' and the content 'All students selected for the RISE Program are required to attend the orientation session on June 15, 2026, at 10:00 AM in Room 509, Haein Hall 4. Attendance is mandatory.' The third card is titled 'Sports Festival Registration' with the date '6/5/2026' and the content 'The registration for Sports Festival 2026 is now open. All international students are welcome to participate. Events include Football, Volleyball, Tug of War, and Mini Games. Please register before June 8, 2026. For inquiries, contact the Event Director.'

Figure 13: Notice Board Interface

vii. Job Portal Interface

The Job Portal provides employment opportunities for students.

Features:

- Job search
- Job filtering
- Job applications
- Application tracking

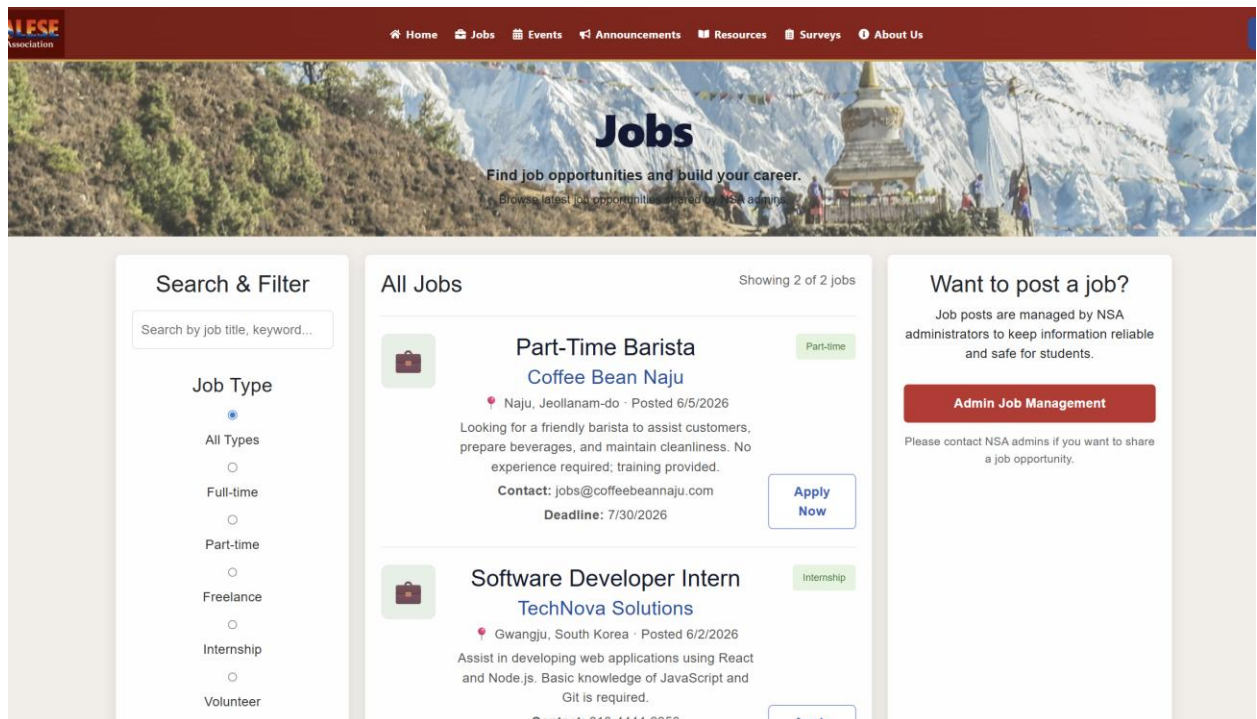


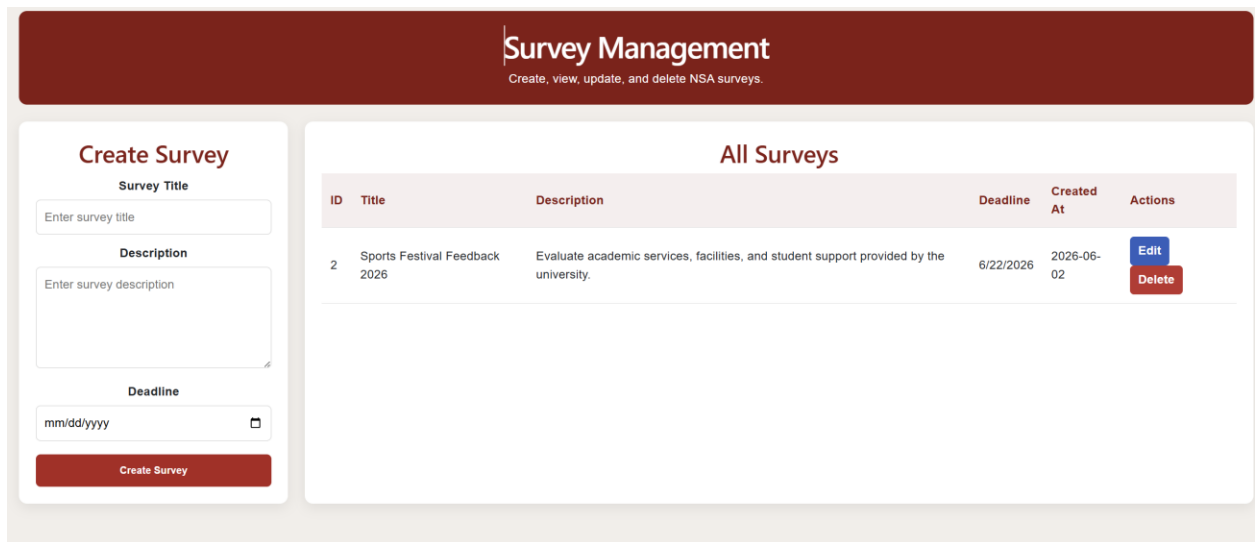
Figure 14: Job Portal Interface

viii. Survey Interface

Students can participate in surveys and administrators can create survey forms.

Features:

- Survey creation
- Survey response collection
- Duplicate submission prevention
- Survey analytics



The screenshot displays the 'Survey Management' interface. At the top, a dark red header contains the title 'Survey Management' and the subtitle 'Create, view, update, and delete NSA surveys.' Below the header, the interface is split into two main sections. On the left, the 'Create Survey' form includes fields for 'Survey Title', 'Description', and 'Deadline' (with a date picker), and a 'Create Survey' button. On the right, the 'All Surveys' section shows a table with one survey entry. The table has columns for ID, Title, Description, Deadline, Created At, and Actions. The entry has ID 2, Title 'Sports Festival Feedback 2026', Description 'Evaluate academic services, facilities, and student support provided by the university.', Deadline '6/22/2026', and Created At '2026-06-02'. The Actions column contains 'Edit' and 'Delete' buttons.

ID	Title	Description	Deadline	Created At	Actions
2	Sports Festival Feedback 2026	Evaluate academic services, facilities, and student support provided by the university.	6/22/2026	2026-06-02	Edit Delete

Figure 15: Survey Interface

Responsive Design

The interface supports:

- Desktop devices
- Laptop devices
- Tablets
- Mobile phones

Bootstrap's responsive grid system ensures consistent display across different screen sizes.

5.2 Static Model – Class Diagrams

The class diagram describes the static structure of the NSA Website system and illustrates the relationships between classes. The design follows object-oriented principles to improve maintainability and scalability. The class diagram is shown in the design document.

Main Classes

i. User Class

The User class stores account information and authentication details.

Attributes:

- user_id
- name
- email
- password
- role
- department
- nationality

Methods:

- register()
- login()
- updateProfile()

ii. Event Class

The Event class manages association activities and schedules.

Attributes:

- event_id
- title
- description

- location
- date

Methods:

- createEvent()
- updateEvent()
- deleteEvent()

iii. Notice Class

The Notice class manages announcements.

Attributes:

- notice_id
- title
- content
- created_at

Methods:

- createNotice()
- updateNotice()
- deleteNotice()

iv. Job Class

The Job class manages employment opportunities.

Attributes:

- job_id
- title
- description
- location

- salary

Methods:

- createJob()
- updateJob()
- deleteJob()

v. Survey Class

The Survey class manages survey forms.

Attributes:

- survey_id
- title
- description

Methods:

- createSurvey()
- updateSurvey()
- deleteSurvey()

vi. SurveyResponse Class

Stores survey responses submitted by students.

Attributes:

- response_id
- survey_id
- user_id
- response_text

Methods:

- submitResponse()

Relationships

- One User can submit many Survey Responses.
- One Survey can contain many Survey Responses.
- One Administrator can create multiple Events.
- One Administrator can publish multiple Notices.
- One Administrator can post multiple Jobs.

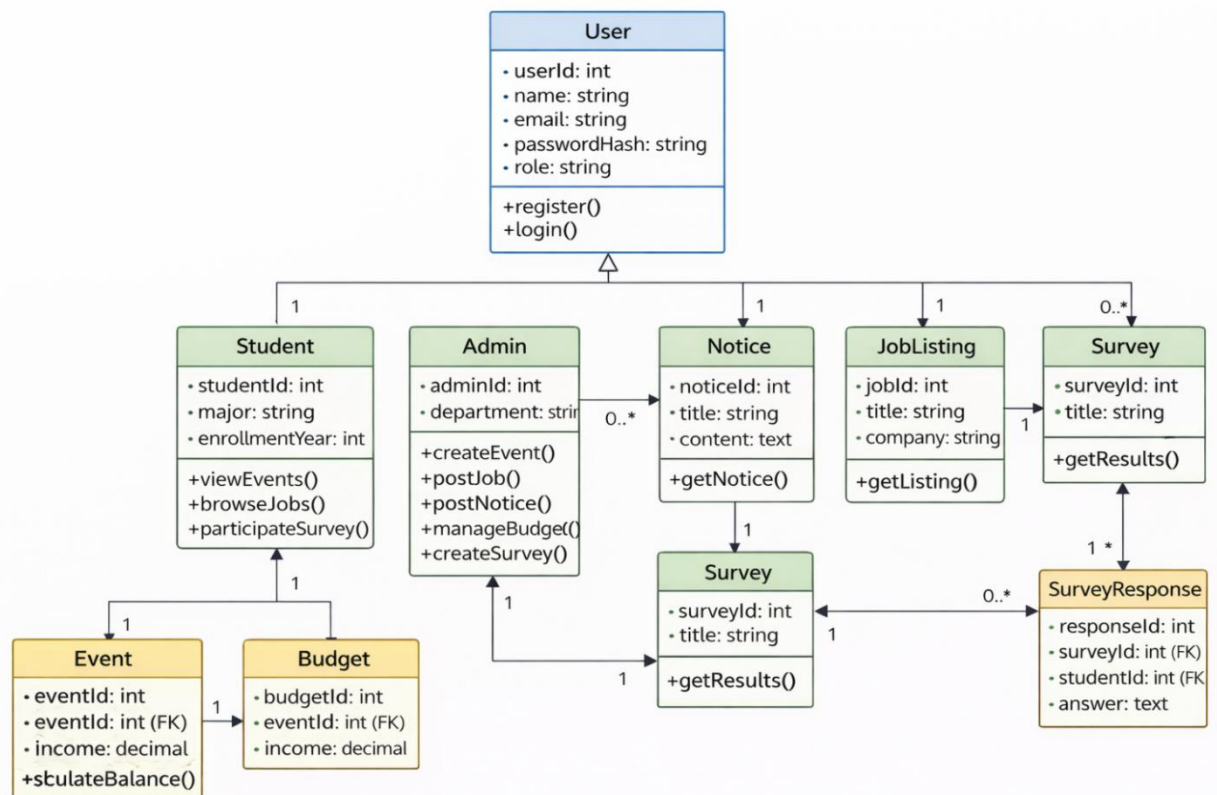


Figure 16: Class Diagram

5.3 Dynamic Model – Sequence Diagrams

Sequence diagrams describe interactions between system components over time.

The NSA Website includes four primary sequence diagrams.

i. User Login Sequence Diagram

The User Login sequence describes the authentication process.

Process Flow

1. User enters credentials.
2. Frontend sends login request.
3. Backend queries database.
4. Database returns user record.
5. Password is verified.
6. JWT token is generated.

7. Login response is returned.
8. Dashboard is displayed.

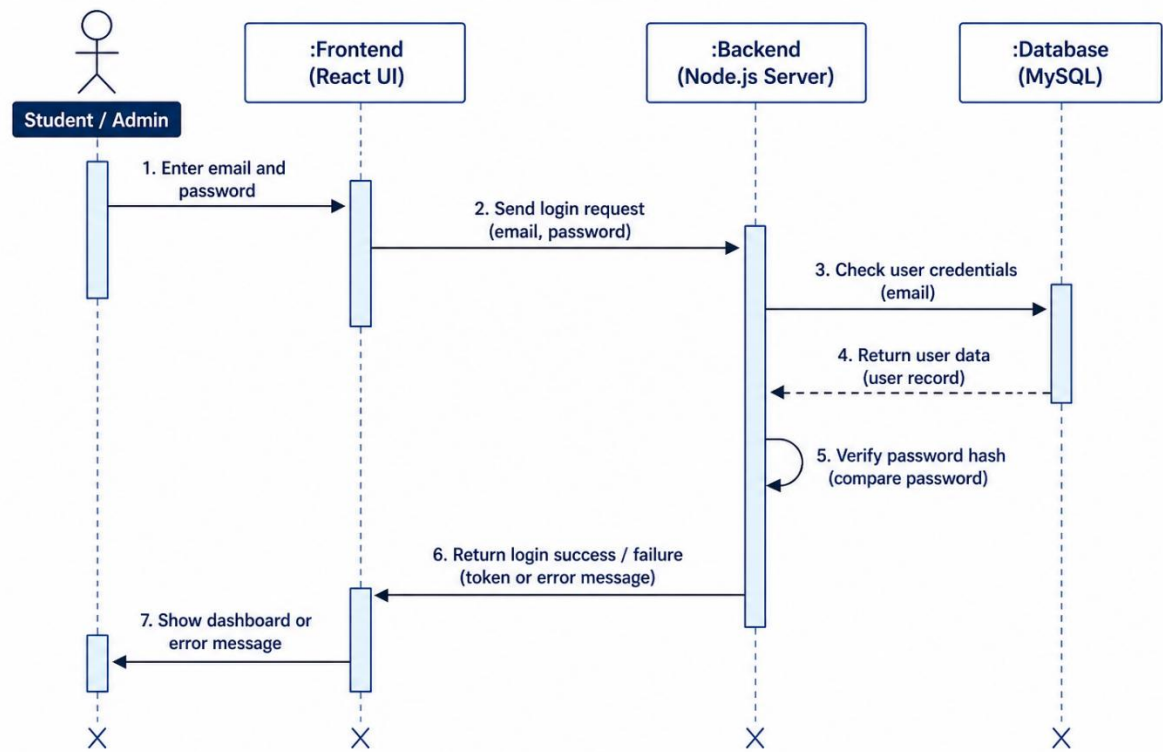


Figure 17: User Login Sequence Diagram

ii. Admin Creates Event Sequence Diagram

This sequence diagram illustrates event creation.

Process Flow

1. Admin enters event information.
2. Frontend submits event request.
3. Backend validates admin role.
4. Event data is inserted into database.
5. Success response is returned.
6. Event becomes available to students.

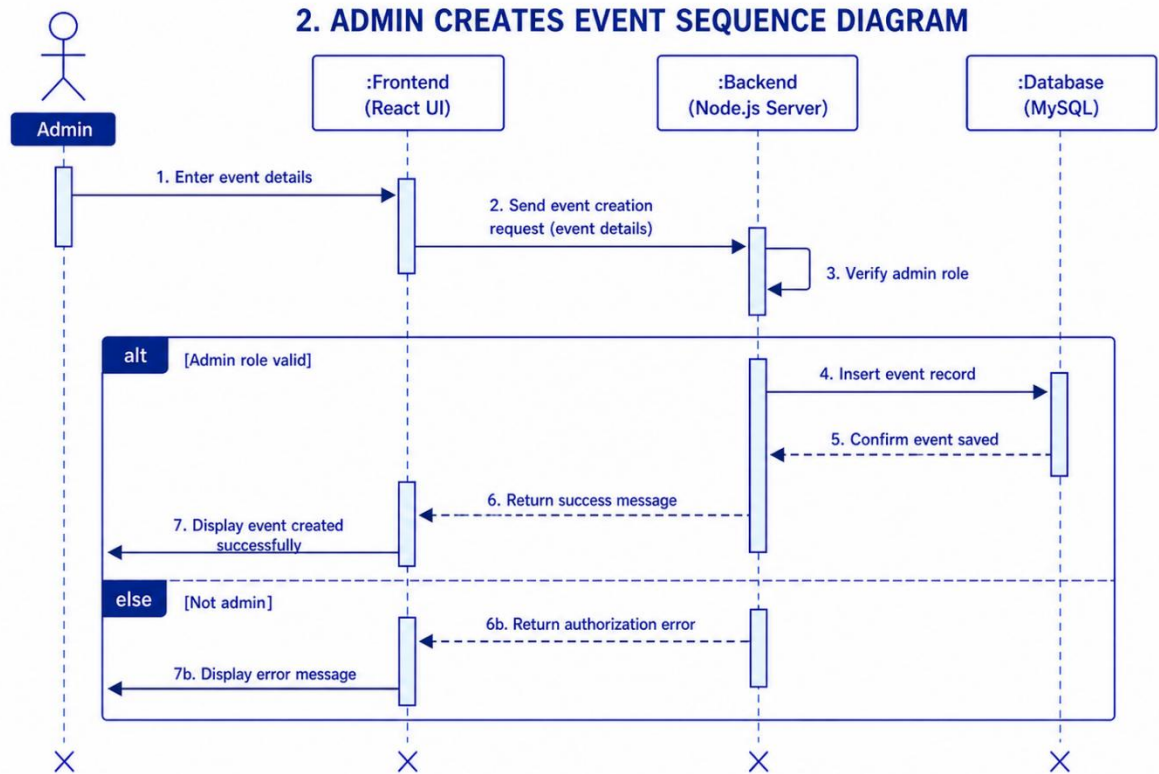


Figure 18: Admin Creates Event Sequence Diagram

iii. Student Submits Survey Sequence Diagram

This sequence diagram shows survey participation.

Process Flow

1. Student selects survey.
2. Student submits responses.
3. Backend checks duplicate submission.
4. Database stores response.
5. Success message is displayed.

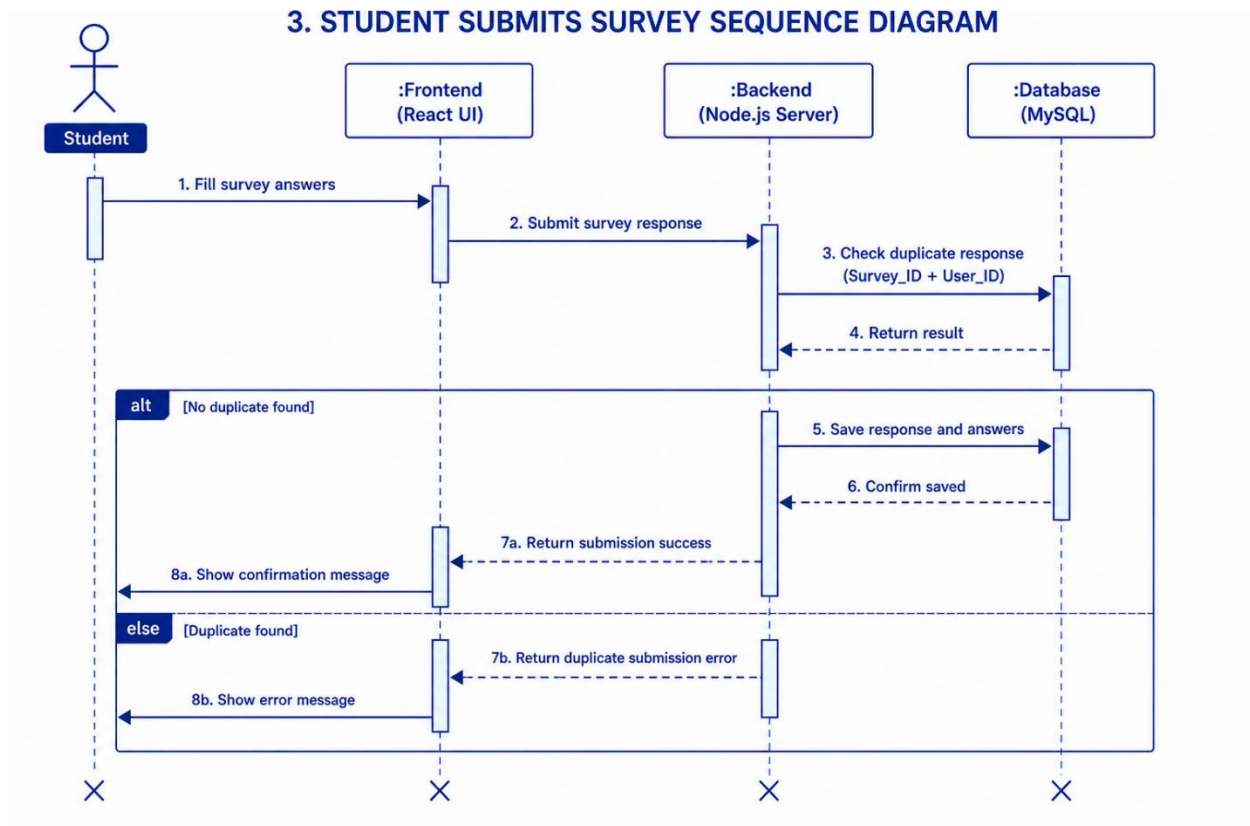


Figure 19: Student Submits Survey Sequence Diagram

iv. Admin Posts Job Sequence Diagram

This sequence diagram describes job posting.

Process Flow

1. Admin enters job details.
2. Backend verifies permissions.
3. Database stores job information.
4. Job becomes visible in Job Portal.

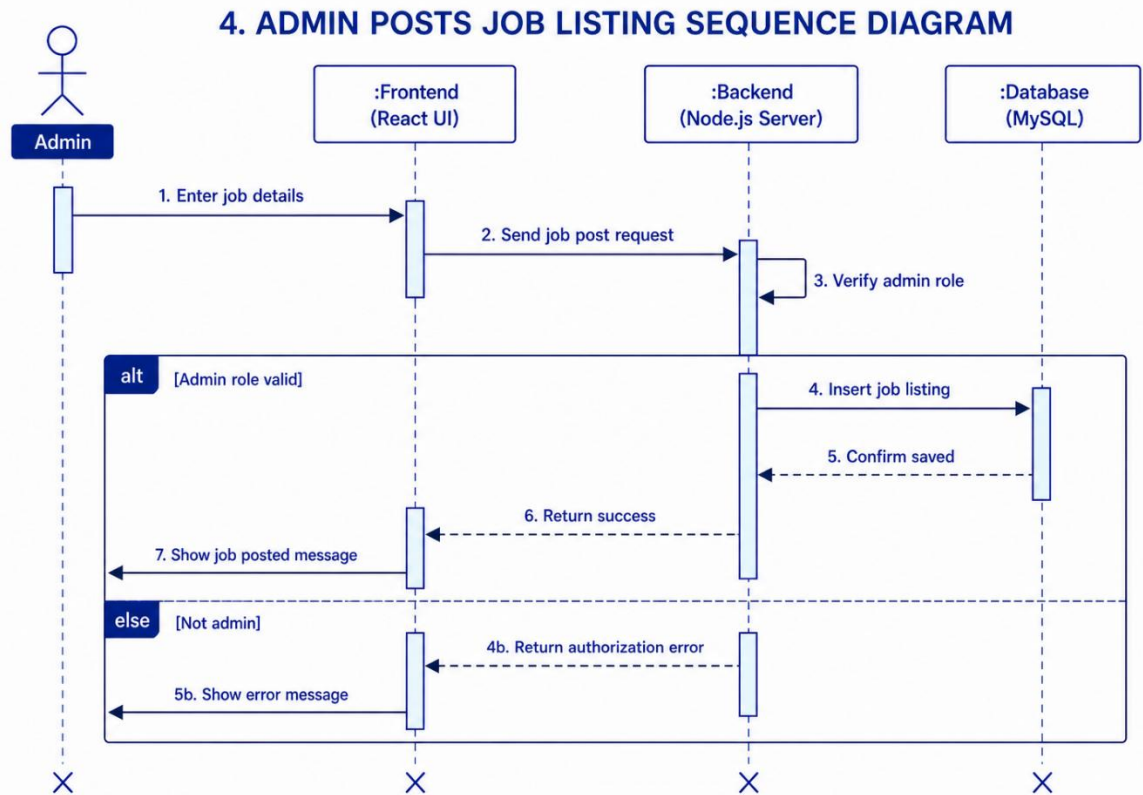


Figure 20: Admin Posts Job Sequence Diagram

5.4 Rationale for Detailed Design Model

The detailed design model was developed to provide a clear blueprint for implementation while ensuring consistency between requirements and system architecture.

Reasons for Using Class Diagrams

Class diagrams:

- Represent system structure.
- Identify relationships between entities.
- Support object-oriented design.
- Improve maintainability.

Reasons for Using Sequence Diagrams

Sequence diagrams:

- Describe system behavior.
- Illustrate communication between components.
- Help identify processing steps.
- Support testing activities.

Reasons for Using GUI Design Models

GUI models:

- Improve user experience.
- Reduce development errors.
- Provide visual guidance during implementation.
- Support stakeholder validation.

The combination of these models ensures that both structural and behavioral aspects of the system are properly documented.

5.5 Traceability from Requirements to Detailed Design Model

The traceability matrix ensures that every functional requirement is represented within the design models.

Requirement	Design Component
REQ-1 Event Management	Event Class, Event Sequence Diagram
REQ-2 Calendar View	Event Module, GUI Design
REQ-3 Job Management	Job Class, Job Sequence Diagram
REQ-4 Job Browsing	Job Portal Interface
REQ-5 Budget Management	Budget Module Design
REQ-6 Budget Viewer	Budget GUI Design
REQ-7 Notice Management	Notice Class
REQ-8 Notice Viewing	Notice Board Interface
REQ-9 Survey Management	Survey Class
REQ-10 Survey Submission	Survey Response Class, Survey Sequence Diagram
REQ-11 Authentication	User Class, Login Sequence Diagram
REQ-12 Registration/Login	Registration Interface
REQ-13 User Management	User Management Module

The traceability matrix demonstrates that every requirement identified during analysis is addressed by one or more design components, ensuring completeness and consistency throughout the development process.

Chapter 6

Test Plan

6.1 Requirements/Specifications-Based System Level Test Cases

Testing was conducted to verify that the NSA Website satisfies the functional and non-functional requirements specified in the Software Requirements Specification (SRS). The testing process focused on validating system functionality, usability, security, and reliability.

Testing Environment

Component	Description
Frontend	React.js
Backend	Node.js + Express.js
Database	MySQL
Deployment	Railway Cloud Platform
Browser	Google Chrome
Testing Tool	Postman
Operating System	Windows 11

Test Case TC-01: User Registration

Item	Description
Test Case ID	TC-01
Requirement	REQ-12
Objective	Verify account registration
Input	Valid registration details
Expected Result	User account created successfully
Actual Result	Account created successfully
Status	Pass

Procedure

1. Open registration page.
2. Enter student information.
3. Request OTP.
4. Verify OTP.
5. Submit registration form.

Result:

The system successfully created a new account and stored user information in the database.

Test Case TC-02: User Login

Item	Description
Test Case ID	TC-02
Requirement	REQ-11
Objective	Verify authentication process
Input	Valid email and password
Expected Result	Successful login
Actual Result	Login successful
Status	Pass

Procedure

1. Enter registered email.
2. Enter password.
3. Submit login request.

Result:

The system generated a JWT token and redirected the user to the dashboard.

Test Case TC-03: Invalid Login

Item	Description
Test Case ID	TC-03
Requirement	REQ-11
Objective	Verify login validation
Input	Invalid password
Expected Result	Error message displayed
Actual Result	Error displayed
Status	Pass

Result:

Unauthorized users were prevented from accessing the system.

Test Case TC-04: Password Reset

Item	Description
Test Case ID	TC-04
Requirement	REQ-11
Objective	Verify password recovery
Input	Registered email
Expected Result	OTP email delivered
Actual Result	OTP received successfully
Status	Pass

Result:

Password reset functionality worked correctly using Brevo email services.

Test Case TC-05: Event Creation

Item	Description
Test Case ID	TC-05
Requirement	REQ-1
Objective	Verify event creation
Input	Event details
Expected Result	Event stored successfully
Actual Result	Event created successfully
Status	Pass

Result:

Event information was stored and became visible to students.

Test Case TC-06: Event Update

Item	Description
Test Case ID	TC-06
Requirement	REQ-1
Objective	Verify event modification
Input	Updated event details
Expected Result	Event updated
Actual Result	Event updated
Status	Pass

Test Case TC-07: Event Deletion

Item	Description
Test Case ID	TC-07
Requirement	REQ-1
Objective	Verify event deletion
Input	Existing event
Expected Result	Event removed
Actual Result	Event removed
Status	Pass

Test Case TC-08: Notice Creation

Item	Description
Test Case ID	TC-08
Requirement	REQ-7
Objective	Verify notice publishing
Input	Notice details
Expected Result	Notice visible to students
Actual Result	Notice displayed
Status	Pass

Test Case TC-09: Notice Editing

Item	Description
Test Case ID	TC-09
Requirement	REQ-7
Objective	Verify notice update

Input	Updated notice
Expected Result	Notice updated
Actual Result	Notice updated
Status	Pass

Test Case TC-10: Job Posting

Item	Description
Test Case ID	TC-10
Requirement	REQ-3
Objective	Verify job posting
Input	Job information
Expected Result	Job appears in portal
Actual Result	Job displayed successfully
Status	Pass

Test Case TC-11: Job Browsing

Item	Description
Test Case ID	TC-11
Requirement	REQ-4
Objective	Verify job browsing
Input	Search keyword
Expected Result	Matching jobs displayed
Actual Result	Jobs displayed
Status	Pass

Test Case TC-12: Survey Creation

Item	Description
Test Case ID	TC-12
Requirement	REQ-9
Objective	Verify survey creation
Input	Survey details
Expected Result	Survey published
Actual Result	Survey created
Status	Pass

Test Case TC-13: Survey Submission

Item	Description
Test Case ID	TC-13
Requirement	REQ-10
Objective	Verify survey participation
Input	Survey response
Expected Result	Response saved
Actual Result	Response stored
Status	Pass

Test Case TC-14: User Authorization

Item	Description
Test Case ID	TC-14
Requirement	REQ-13
Objective	Verify role-based access

Input	Student attempting admin action
Expected Result	Access denied
Actual Result	Access denied
Status	Pass

6.2 Traceability of Test Cases to Use Cases

The following matrix demonstrates the relationship between use cases and corresponding test cases.

Use Case	Test Case
Registration	TC-01
Login	TC-02, TC-03
Password Reset	TC-04
Manage Events	TC-05, TC-06, TC-07
Manage Notices	TC-08, TC-09
Manage Job Listings	TC-10
Browse Jobs	TC-11
Manage Surveys	TC-12
Submit Survey	TC-13
Manage Users	TC-14

This traceability ensures that all major use cases are validated through testing activities.

6.3 Techniques Used for Test Generation

Several testing techniques were used throughout the project.

Black Box Testing

Black box testing was used to verify system functionality without examining internal implementation details.

Examples:

- Registration testing
- Login testing
- Event creation testing
- Job posting testing

Functional Testing

Functional testing verified whether implemented features satisfied specified requirements.

Examples:

- Authentication
- Survey participation
- Notice publishing
- Event management

Integration Testing

Integration testing ensured proper communication between system modules.

Examples:

- React frontend ↔ Express backend
- Backend ↔ MySQL database
- Backend ↔ Brevo Email API

User Acceptance Testing (UAT)

User Acceptance Testing was performed to ensure the system meets stakeholder expectations.

Focus areas:

- Ease of use
- Navigation
- User experience
- Information accessibility

6.4 Assessment of the Goodness of the Test Suite

The quality of the test suite was evaluated using several software testing metrics.

Requirement Coverage

Requirement coverage measures how many system requirements are tested.

Requirement Type	Coverage
Functional Requirements	100%
Non-Functional Requirements	80%

Requirement coverage was achieved through systematic mapping between requirements and test cases.

Use Case Coverage

Use Cases Tested	Coverage
12 of 12 Major Use Cases	100%

All major user interactions were tested.

Pass Rate

Metric	Value
Total Test Cases	14
Passed	14
Failed	0
Success Rate	100%

Defect Detection

Several defects were identified and corrected during development, including:

- Database connection errors
- Railway deployment issues
- Package dependency conflicts
- OTP email delivery failures
- Authentication validation bugs

After corrective actions, the system passed all critical functional tests.

Reliability Assessment

The deployed system demonstrated:

- Stable cloud deployment
- Successful database connectivity
- Secure authentication
- Consistent API responses

The Railway deployment environment further improved reliability through managed hosting services.

Overall Testing Evaluation

The testing process confirmed that the NSA Website satisfies the majority of its specified requirements and performs reliably under normal operating conditions. The combination of functional, integration, and user acceptance testing provides confidence in the system's readiness for deployment and practical use by the Nepalese Student Association.

Acknowledgment

The project team would like to express sincere gratitude to **Prof. Ibrahim Musa Ishag Musa** for his valuable guidance, continuous support, and constructive feedback throughout the development of this project. His expertise and encouragement played a significant role in the successful completion of the NSA Website.

The team would also like to thank **Dongshin University** and the **College of Global Autonomous Major** for providing the resources, facilities, and academic environment necessary for completing this Software Engineering project.

References

- [1] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.
- [2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.
- [3] IEEE Computer Society, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.
- [4] React Team, "React Documentation," Meta Platforms Inc. [Online]. Available: <https://react.dev>. [Accessed: Jun. 8, 2026].
- [5] Node.js Foundation, "Node.js Documentation." [Online]. Available: <https://nodejs.org>. [Accessed: Jun. 8, 2026].
- [6] Express.js, "Express Web Framework." [Online]. Available: <https://expressjs.com>. [Accessed: Jun. 8, 2026].
- [7] Oracle Corporation, "MySQL Documentation." [Online]. Available: <https://dev.mysql.com/doc>. [Accessed: Jun. 8, 2026].
- [8] Railway, "Railway Documentation." [Online]. Available: <https://docs.railway.app>. [Accessed: Jun. 8, 2026].
- [9] GitHub Inc., "GitHub Documentation." [Online]. Available: <https://docs.github.com>. [Accessed: Jun. 8, 2026].
- [10] Bootstrap Team, "Bootstrap Documentation." [Online]. Available: <https://getbootstrap.com>. [Accessed: Jun. 8, 2026].
- [11] Axios Contributors, "Axios Documentation." [Online]. Available: <https://axios-http.com>. [Accessed: Jun. 8, 2026].
- [12] Brevo, "Brevo Email API Documentation." [Online]. Available: <https://developers.brevo.com>. [Accessed: Jun. 8, 2026].

Appendix A – Project Resources

GitHub Repository:

<https://github.com/sabin2002/NSA-website>

Live System URL:

<https://handsome-generosity-production-8a66.up.railway.app/>

Backend API URL:

<https://nsa-website-production.up.railway.app>

Deployment Platform:

Railway Cloud Platform

Database:

Railway MySQL Database